Lab1-算术与逻辑运算指令

- 姓名：戴益阳
- 专业：计算机科学与技术
- 学号：24300240114

- 实验时间：2026/3/3 ~ 2025/3/15
- 指导老师：陈辰
- 本次实验，我完成了所有内容。

目录

- Lab1-算术与逻辑运算指令
  - 目录
  - 实验目的
  - 实验内容
    - 1. RISC-V指令格式
    - 2. 解决数据冲突问题（Data Hazard）
    - 3. IO总线与步进信号
    - 4. Difftest接线
    - 5. 使用verilator工具实现CPU的设计
  - 实验结果
  - 遇到的问题及解决办法
  - 实验心得
  - 其他备注

实验目的

1. 理解RISC-V指令集的编码格式与执行流程，掌握指令分类及其控制信号生成机制。
2. 掌握流水线CPU的数据通路与控制单元设计方法，实现指令的取指、译码、执行、访存、写回的五级流水。
3. 掌握带有IO总线的CPU的设计方法，实现取指时CPU与IO总线的交互，并用全局的step信号和各个模块的ok信号来控制步进。
4. 掌握使用Difftest工具对CPU进行测试，学会将CPU中的寄存器等接口暴露给Difftest工具，并在正确的时候发出提交信号。
5. 掌握使用Linux环境下的verilator工具对systemverilog语言文件进行仿真，了解verilator工具与vivado使用时的异同。
6. 掌握使用gtkwave工具观察波形图，并进行代码调试

实验内容

本次实验要求实现能够实现如下指令的带有IO总线的64位RISC-V流水线CPU：

算术运算与逻辑运算：

addi, xori, ori, andi

add, sub, and, or, xor

addiw, addw, subw

1. RISC-V指令格式

通过查表可知，本次实验要实现的指令格式如下：

```
0000000 rs2 rs1 000 rd 0110011 R add
0100000 rs2 rs1 000 rd 0110011 R sub
0000000 rs2 rs1 111 rd 0110011 R and
0000000 rs2 rs1 110 rd 0110011 R or
0000000 rs2 rs1 100 rd 0110011 R xor

imm[11:0] rs1 000 rd 0010011 I addi
imm[11:0] rs1 111 rd 0010011 I andi
imm[11:0] rs1 110 rd 0010011 I ori
imm[11:0] rs1 100 rd 0010011 I xori

imm[11:0] rs1 000 rd 0011011 I addiw

0000000 rs2 rs1 000 rd 0111011 R addw
0100000 rs2 rs1 000 rd 0111011 R subw
```

其中，R型指令的计算方式为：

rd = rs1 op rs2

I型指令的计算方式为：

rd = rs1 op (SignExtended) imm

后缀有w的指令操作为对两个32位的数进行运算，最后再符号扩展至64位，目的是为了使得程序在64位的操作系统上与旧的32位操作系统兼容。

2. 解决数据冲突问题（Data Hazard）

本次实验中由于只涉及寄存器内的操作，没有load-use和跳转的情况，因此可以只通过转发（Forword or Bypass）的方法来解决数据冲突问题（Data Hazard），即从memory和execute模块将信号传回D到E阶段的寄存器进行判断，当memory和execute模块中的指令中有一个目的寄存器与存储器阶段的源寄存器相匹配时，则进行替换，还需要注意目的寄存器必须不为$0寄存器。

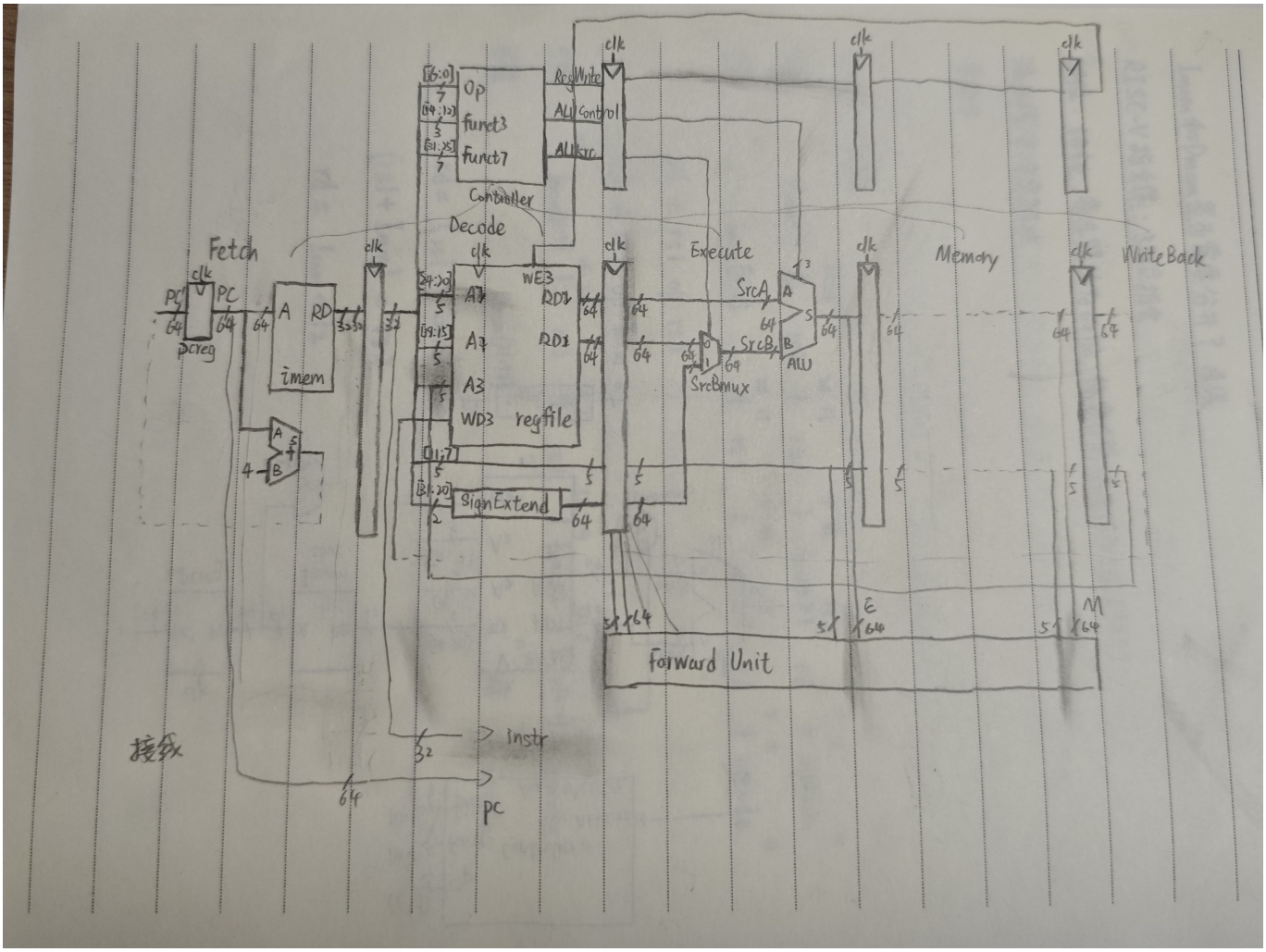
3. IO总线与步进信号

本次实验要实现带有IO总线的CPU，因此CPU需要能够与IO总线进行交互，即实现发出请求，等待，获取数据三步，这个过程需要靠全局的step信号以及各个模块的ok信号来控制，当所有模块的ok信号都为1时，step为1，各个模块再同时进行步进，因此，带有IO总线的CPU无法以简单的单周期方式实现，在以流水线的方式实现时，也需要多个时钟周期才能进行一次步进。

4. Difftest接线

本次实验需要通过Difftest工具来进行测试CPU，因此需要将额外的接口暴露到外层，例如寄存器，还需要额外将pc信号和instr信号同步向前传播，在提交指令时，需要过滤掉一开始的几个步进，一直等到信号步进到writeback模块时才能进行第一次提交，而且由于提交信号与写入寄存器的操作同步进行，所以需要通过组合逻辑先对寄存器的值进行一次保存。

最后得到的设计图如下：



5. 使用verilator工具实现CPU的设计

在使用verilator工具时，需要手动include调用的模块，而不是像vivado一样会自动管理各个模块之间的调用关系。

核心代码有：

forward部分：

```
// forward

logic [63:0] true_readData1D, true_readData2D;

logic [4:0] sreg1, sreg2;
```

```
assign sreg2 = instrD[24:20];
assign sreg1 = instrD[19:15];

always_comb begin
    true_readData1D = readData1D;
    true_readData2D = readData2D;
    if (regwriteM && writeRegM != 0) begin
        if (writeRegM == sreg1) true_readData1D = aluresultM;
        if (writeRegM == sreg2) true_readData2D = aluresultM;
    end
    if (regwriteE && writeRegE != 0) begin
        if (writeRegE == sreg1) true_readData1D = aluresultE;
        if (writeRegE == sreg2) true_readData2D = aluresultE;
    end
end
```

decode部分：

```
`ifdef VERILATOR
`include "include/common.sv"
`endif

module decode import common::*;(
    input  logic clk, reset,
    input  logic step, // 这个信号用来同步整个 CPU 的时序，当其为 1 时，整个 CPU 流水线向前移动一个指令。
    output logic decode_ok, // 表示当前模块是否已经准备好接受下一条指令了
    // 实际上 step = fetch_ok & decode_ok & execute_ok & memory_ok & writeback_ok; 也就是说，只有当五个阶段都准备好接受下一条指令了，step 才会为 1。
    input  logic [31:0] instr,
    input  logic regwrite,
    input  logic [4:0] writeAddr3,
    input  logic [63:0] writeData3,
    output logic [63:0] readData1,
    output logic [63:0] readData2,
    output logic [4:0] writeRegD,
    output logic [63:0] seimm,
    output logic [63:0] next_reg[31:0]); // 输出寄存器

logic [63:0] rf[31:0]; // 主寄存器

logic [4:0] readAddr1;
logic [4:0] readAddr2;

logic [11:0] imm;

assign readAddr2 = instr[24:20];
assign readAddr1 = instr[19:15];

assign imm = instr[31:20];

assign readData1 = (readAddr1 == 0) ? 0 : next_reg[readAddr1];
```

```
assign readData2 = (readAddr2 == 0) ? 0 : next_reg[readAddr2];
assign writeRegD = instr[11:7];
assign seimm = {{52{imm[11]}}, imm};

always_comb begin
    for (int i = 0; i < 32; i++) begin
        if (regwrite && (i != 0) && (i[4:0] == writeAddr3)) begin
            next_reg[i[4:0]] = writeData3; // 用组合逻辑向next_reg写入
        end else begin
            next_reg[i[4:0]] = rf[i[4:0]]; // 复制其他没有写入的寄存器
        end
    end
end

always_ff @(posedge clk) begin
    if (reset) begin
        decode_ok <= 1;
    end else if (step) begin
        decode_ok <= 0; // 先把 decode_ok 置为 0，表示我们正在处理当前指令，还没有准备
好接受下一条指令了。
        if (regwrite && writeAddr3 != 0) rf[writeAddr3] <= writeData3;
    end else begin
        // 这里对应：要么我们还没译码好指令，要么我们译码好指令了，在等其他模块
        if (decode_ok) begin
            // 在等其他模块
        end else begin
            decode_ok <= 1;
        end
    end
end

endmodule
```

Difftest接线部分：

```
assign valid = step & (instrW != 0);
```

实验结果

```
TEST= ./build/emu --diff /home/dyy/arch_2026/26-Arch/ready-to-run/riscv64-nemu-interpreter-so -i ./ready-to-run/lab1/lab1-test.bin --dump-wav
e || true
Emu compiled at Mar 10 2026, 17:16:55
The image is ./ready-to-run/lab1/lab1-test.bin
Using simulated 256MB RAM
Using /home/dyy/arch_2026/26-Arch/ready-to-run/riscv64-nemu-interpreter-so for difftest
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
dump wave 0-999999 to /home/dyy/arch_2026/26-Arch/build/1773560994.fst...
The first instruction of core 0 has commited. Difftest enabled.
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 3210 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 5104361 instr/s
No instruction of core 0 commits for 5000 cycles, maybe get stuck
(please also check whether a fence.i instruction requires more than 5000 cycles to flush the icache)
Let REF run one more instruction.
Program execution has ended. To restart the program, exit NEMU and run again.
sh: 1: spike-dasm: not found
```

遇到的问题及解决办法

1. 遇到问题： 冲突的解决方法是用阻塞还是转发？只有寄存器操作是否还需要memory模块？

   解决方案：暂时先使用了转发方法，性能更好，也相对容易些；仍旧保留了memory模块，尽管memory模块暂时没有任何作用，以便于下次实验进行扩展。

2. 遇到问题： 注意RISC-V指令高5位[24:20]和低5位[19:15]分别对应rs2和rs1，计算时左元为rs1，右元为rs2或立即数，即rs1 op rs2/imm

   解决方案：decode模块需要仔细接线。

3. 遇到问题： 仿真时报错：找不到datapath.sv

   解决方案：verilator需要手动include调用的模块

4. 遇到问题： Difftest接线的valid信号怎么接

   解决方案：尝试了各种方法，最后通过观察波形图的方式选择了直接让valid信号等于步进的step信号，并通过instrW不为0的方式过滤掉开始的几个步进信号，后续也可以通过这个方式扩展到阻塞的情况。

5. 遇到问题： Difftest接线的寄存器信号怎么接

   解决方案： 创建一个使用组合逻辑实现的寄存器副本，既解决了writeback阶段指令的转发问题，又解决了写入寄存器和提交指令信号同时发生的问题

6. 遇到问题： 存在写入$0寄存器的违法指令

   解决方案： 在decode模块中进行判断并排除

实验心得

这次实验让我学会了带有IO总线的流水线CPU的实现方法，掌握了在verilator环境和Difftest要求下的实现方法，为后续增加更多指令奠定了基础。

其他备注

无